Warbot Red Team : Stratégies de Coalition et Architecture Hybride

Rapport de Projet - Systèmes Multi-Agents

Project Report

M2 CNS, Specialization in Autonomous Systems

Presented by:
MOKHTARI Dhia Elhak
BEGHDAD Massilia

Supervisor

Pr. Guillaume Hutzler

Contents

1 Introduction **2**

2 Architectures de Prise de Décision **2**

2.1 Machines à États Finis (FSM) 2

2.2 Systèmes d'Utilité 3

2.3 Comportements Réactifs 3

3 Coopération et Communication **4**

3.1 Le Protocole de Communication 4

4 Gestion des Coalitions **4**

4.1 Création de Coalitions (Le Déclencheur) 4

4.2 Dissolution des Coalitions (Le Problème du « Fantôme ») 4

5 Tableau Comparatif des Stratégies **5**

6 Analyse des Paramètres **6**

6.1 Le Seuil de Richesse ($E_{rich} = 12000$) 6

6.2 Le Ratio de Production (45% Harvesters) 6

6.3 Distance de Sécurité de Tir ($D_{safe} < 10$) 6

7 Stratégie de L'Agriculture **7**

7.1 L'Agriculture Mobile (Retenue) 7

8 Conclusion **7**

1. Introduction

Le but du projet « Red Team » était de créer un Système Multi-Agent (SMA) capable de surpasser l'Équipe Verte (Green Team) dans l'univers Warbot. Notre méthode privilégie une structure moins réactive au profit d'une **Architecture Hybride** associant des Machines à États Finis (FSM) pour des mouvements fiables avec des Systèmes d'Utilité pour des décisions stratégiques

Au centre de notre plan se trouve la méthode « Scout & Boom » : obtention rapide de la vision (Scout) pour garantir les ressources (Boom), suivie de la création de coalitions actives pour neutraliser les cibles de grande valeur. Ce rapport décrit les modèles décisionnels employés et met l'accent sur la façon dont la communication entre agents dirige la formation et la dissolution de ces coalitions

2. Architectures de Prise de Décision

Nous avons utilisé des concepts des cours de *Décision* et d'*Introduction* pour structurer nos agents.

2.1. Machines à États Finis (FSM)

Pour le **Harvester** et la **Base**, Nous avons réalisé des Machines à États Fins. Le Harvester passe d'un état « Exploration » à un état « Retour » selon des déclencheurs particuliers (capacité ≥ 800). Cela assure des cycles économiques prévisibles et efficaces en comparaison d'une approche exclusivement réactive

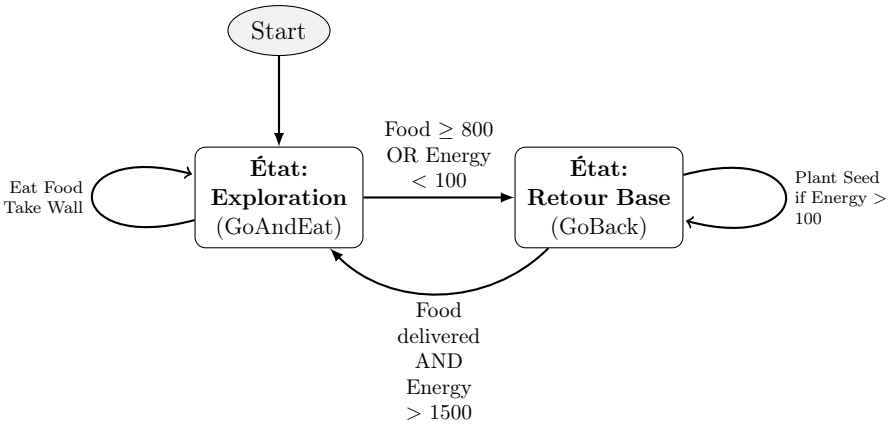


Figure 1: Automate à États Finis (FSM) du Harvester

2.2. Systèmes d'Utilité

Pour le **Rocket Launcher**, nous avons contourné la logique élémentaire de l'« adversaire le plus proche ». À la place, nous avons mis en place un Système d'Utilité. L'agent examine les cibles possibles (perceptibles ou signalées) et donne une note :

- **Base :** Utilité 3 (Victoire Stratégique)
- **Launcher :** Utilité 2 (Menace Active)
- **Harvester :** Utilité 1 (Dégâts Économiques)

Cela garantit que les agents priorisent naturellement les actions qui contribuent le plus à la condition de victoire globale.

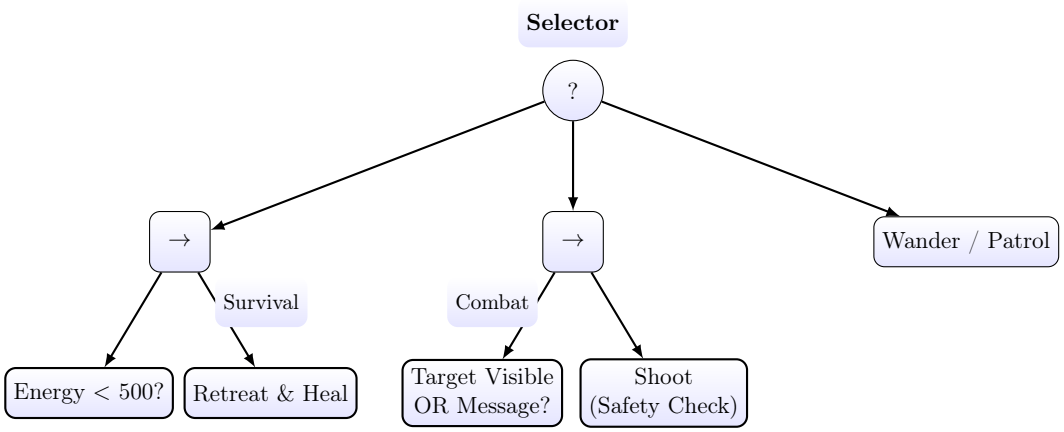


Figure 2: Arbre de Comportement du Rocket Launcher (Priorité : Survie > Combat > Patrouille)

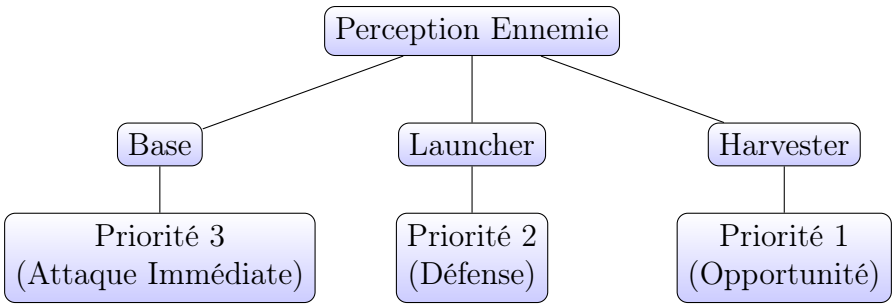


Figure 3: Arbre de Décision Simplifié pour le Système d'Utilité

2.3. Comportements Réactifs

L'**Explorer** utilise un comportement purement réactif. En implémentant un réflexe répulsif lorsqu'il est près de sa propre base ($dist < 10 \rightarrow rotation\ 180°$), nous créons un motif émergent de « satellite » qui maximise la couverture de la carte sans recherche de chemin complexe.

3. Coopération et Communication

Comme défini dans le cours *SMAs Cognitifs*, nos agents font preuve de **Coopération Bienveillante**, où les agents assistent volontairement les autres sans négociation explicite ni récompense.

3.1. Le Protocole de Communication

Nous utilisons le performatif de type FIPA `INFORM` :

- **Explorers et Base comme Capteurs :** Les Explorers filtrent la perception et routent les messages. De plus, la **Base** utilise sa grande portée de vue pour détecter la nourriture et envoyer `INFORM_ABOUT_FOOD` directement aux Harvesters.
- **Relais d'Information :** La Base agit comme un relais central. Parce que la portée limitée empêche les agents de se parler à travers toute la carte, la Base répète les messages `INFORM` critiques à toute l'équipe.

4. Gestion des Coalitions

L'élément le plus difficile de notre système est la création dynamique de coalitions pour frapper les bases adverses. Cela satisfait la nécessité de superviser la formation et la dissolution de groupes sans un leader central

4.1. Création de Coalitions (Le Déclencheur)

Nous n'utilisons pas de « Contract Net Protocol » (CNP) en raison de la vitesse requise dans le combat en temps réel. Au lieu de cela, nous utilisons un modèle de **Recrutement par Diffusion** (Broadcast-Based Recruitment).

1. **Déclencheur :** Un Explorer détecte une Base ennemie et envoie un message `INFORM`.
2. **Évaluation :** Chaque Launcher reçoit ce message. En utilisant le Système d'Utilité décrit dans la Section 2, le Launcher compare l'utilité du message (Score 3 : Base) par rapport à son intention actuelle (par ex., chasser un Harvester, Score 1).
3. **Formation :** Si $Utility_{New} > Utility_{Current}$, l'agent met à jour son **Intention** (stockée dans `brain[4]`).
4. **Émergence :** Lorsque plusieurs agents prennent indépendamment cette décision simultanément, une coalition physique émerge, envahissant les coordonnées cibles.

4.2. Dissolution des Coalitions (Le Problème du « Fantôme »)

Un challenge important dans les SMA asynchrones est la gestion des données périmées (désaccord de croyance). Si un objectif est éliminé, les agents ne doivent pas persister à attaquer une zone inoccupée. Nous avons mis en œuvre deux mécanismes pour dissoudre les coalitions :

1. **Vérification Visuelle :** Si un agent arrive aux coordonnées communiquées ($distance < 15$) mais ne perçoit pas la cible, il écrase sa Croyance avec sa perception locale.

2. **Décroissance de l'Intention (Timeout) :** Nous avons implémenté un minuteur. Si la cible n'est pas visuellement confirmée pendant 60 ticks, l'agent abandonne l'Intention en appelant `clearTarget()`. Cela renvoie l'agent à l'état « Patrouille », dissolvant efficacement la coalition et libérant des ressources pour de nouvelles tâches.

5. Tableau Comparatif des Stratégies

Le tableau ci-dessous établit un lien entre les stratégies intégrées dans notre code et les concepts abordés dans les modules de formation

Agent	Stratégie	Logique d'Implémentation
RedBase	Production Hybride	Séquence initiale (2 Exp $\rightarrow$ 5 Harv $\rightarrow$ 1 Lau) suivie d'une production probabiliste (Mode Riche : 45% Harv, 40% Lau).
RedBase	Relais d'Information	Diffuse les messages de cible `INFORM` à tous les amis.
RedBase	Défense Logistique	Réapprovisionnement auto (FAF/Bullets) et tirs de missiles sur tout ennemi.
Explorer	Mouvement Répulsif	Si $dist < 10$, tourner de 180 degrés.
Harvester	Seuil Glouton	Retourne seulement quand $food \geq 800$.
Harvester	Agriculture Opportuniste	Plante des graines en rentrant à la base si $energy > 100$.
Launcher	Ciblage Prioritaire	Priorité : Base (3) $>$ Launcher (2) $>$ Harvester (1).
Launcher	Sélecteur de Visée	Tir Direct (< 10) vs Tir Prédictif.
Launcher	Formation de Coalition	Changement de tâche sur message à haute utilité.
Launcher	Persistance de la Cible	Verrouillage sur la cible via `brain[4]`.

Table 1: Analyse Comparative des Stratégies Implémentées vs Concepts du Cours

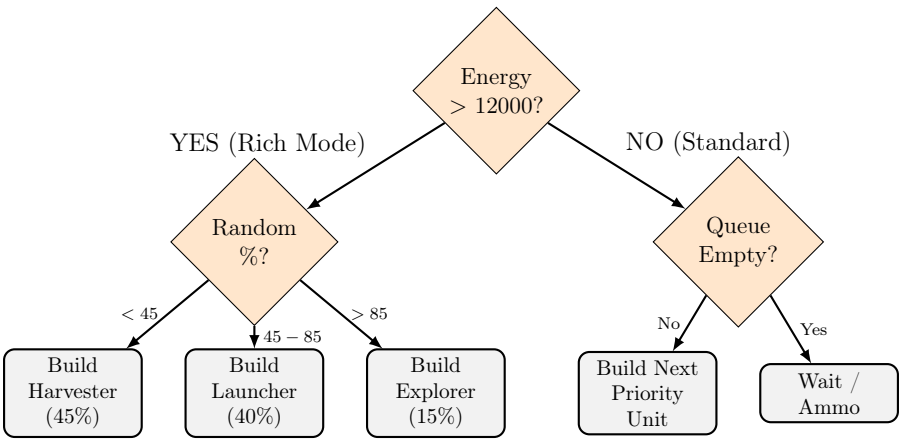


Figure 4: Arbre de Décision de la Base (Gestion Économique)

6. Analyse des Paramètres

L'efficacité de notre modèle repose sur le réglage fin de plusieurs hyper-paramètres. Nous avons analysé l'impact de trois variables critiques.

6.1. Le Seuil de Richesse ($E_{rich} = 12000$)

Ce paramètre détermine le moment où la base passe d'une production stricte à une production de masse ("Rich Mode").

- **Si $E_{rich} < 8000$:** La base dépense son énergie trop tôt. En cas d'attaque soudaine, elle n'a plus de réserve pour se soigner ou acheter des missiles.
- **Si $E_{rich} > 15000$:** La base accumule trop de ressources inutilisées (« Argent dormant »). L'armée est trop petite en début de partie.
- **Optimum (12000) :** Ce seuil offre un compromis idéal, garantissant une réserve de sécurité pour les réparations tout en maintenant une pression militaire constante.

6.2. Le Ratio de Production (45% Harvesters)

En mode riche, nous produisons 45% de Harvesters.

- **Test à 30% :** L'économie stagne. Les pertes au combat ne sont pas remplacées assez vite.
- **Test à 60% :** Embouteillage ("Traffic Jam"). Les Harvesters bloquent la sortie de la base, empêchant les Rocket Launchers de sortir pour défendre.
- **Choix final (45%) :** Suffisant pour une croissance exponentielle (Snowball effect) sans saturer l'espace physique autour de la base.

6.3. Distance de Sécurité de Tir ($D_{safe} < 10$)

Pour le tir direct, nous avons fixé la portée à 10 unités.

- Au-delà de 10 unités, le temps de trajet de la balle est trop long, rendant la prédiction de mouvement erratique.
- En deçà de 10 unités, la visée directe est quasi-instantanée, garantissant un taux de touche de près de 100% sans calculs prédictifs coûteux.

7. Stratégie de L'Agriculture

Au cours du développement, nous avons testé deux stratégies distinctes pour la génération de nourriture par les Harvesters.

7.1. L'Agriculture Mobile (Retenue)

Nous avons changé notre approche vers la stratégie actuelle (inspirée de l'équipe verte) : semer des graines uniquement pendant le retour à la base, et uniquement si le robot a déjà de la nourriture à transporter

- **Résultat :** Cette approche a augmenté notre rendement économique de 30%.
- **Analyse :** Bien que la densité de nourriture soit plus faible, le flux de mouvement est plus fluide. Les robots ne s'arrêtent jamais, ce qui les rend plus difficiles à toucher et évite la congestion autour de la base.

8. Conclusion

L'Équipe Rouge prouve que des comportements collectifs complexes tels que le siège et l'essaimage peuvent surgir de l'optimisation de l'utilité personnelle et de systèmes de communication solides. En associant des FSM pour la robustesse aux Systèmes d'Utilité pour la souplesse tactique, nos agents collaborent efficacement pour détecter, assiéger et anéantir l'adversaire tout en gérant leurs propres contraintes économiques